GITLAB CE · TEST AUTOMATION

GitLab CE 测试框架完整学习计划

设计原理 · 10 周路线 · 8 个阶段 · 38 项验收

API / UI / 性能 / 安全 / 回归 / CI-CD

打印日期：2026 年 10 月 23 日

完成进度：0 / 38 项

每个阶段先读「核心认知」再动手——它讲的是为什么。
验收项全部勾选后再进入下一阶段；勾选状态保存在本机浏览器。

这份计划怎么用

计划不是从零写框架，而是**在你已经建好的 `gitlab-qa` 上加厚。** 现有基线：**88 条用例、7 个分层、10 个套件、12 个 marker**、CI 双跑。 W7 安全层与 W8 回归治理层已落地——后者是本轮新建的，它补的是最容易被跳过的一环： **一轮跑绿说明不了什么，能重复 N 轮都绿才叫可信。**

> **往下之前先看「设计原理」**
> **设计原理** 一节讲的是这套框架**为什么长这样**： 六层结构、八条原则、一条用例的完整链路、几个主动取舍。 它不新增任务，但决定了后面每个阶段你做的是"照抄步骤"还是"理解后自己判断"。

当前基线（可直接核对）

分层	用例	职责	什么时候跑
`unit`	17	框架自检，不联网——框架自己也得有测试	每次改动
`smoke`	4	环境活着 + 核心只读接口	PR 门禁
`api`	22	鉴权 / CRUD / 契约 / 分页	PR 门禁
`ui`	3	Playwright 关键路径	合并前
`e2e`	1	API 造数据 → UI 验收	合并前
`perf`	3	并发采样 + P95 门禁	夜间
`security`	**32**	越权 6 / 注入转义 12 / 泄露 8 / 暴露面 6	夜间 + 发版前

合计 82 条用例函数（参数化展开后 88 项）。实测记录： `--suite security` 32 passed / 0 skipped； 全量 85 passed / 1 skipped，两条性能用例在 4 个 worker 并行时 P95 冲到 3s+ 超阈值， 单独跑 `--suite perf` 3 passed——这正是"性能不进 PR 门禁"的现场证据。
靶机全量（ `--mock` ）80 passed / 8 skipped：8 条 skip 全是"缺第二身份"， 既不算失败也不算通过——**这正是 W8 要解决的"skip 伪装成稳定"。**

节奏安排

W0 · 半天
地基
部署 GitLab CE 靶场，跑通一键脚本

W1–W2
框架搭建
pytest 工程化、夹具分层、数据工厂、失败留证

W3–W4
API 测试
客户端、鉴权状态机、契约三层、数据依赖

W5
UI 测试
POM、通道回退、trace 留证、断言有效性

W6
性能
并发采样、P95 门禁、证明门禁有效

W7
安全 · 已建成
越权 6 / 注入转义 12 / 泄露 8 / 暴露面 6

W8
回归治理 · 已建成
四类稳定性判定、隔离区收费制、重试上报

W9
CI/CD
双平台跑、趋势累积、报告发布、门禁阻断

W10

面试转化

把工程经历讲成能力证据

> **学习顺序的关键**
> 先**建靶场**再建框架。很多人的顺序反了：先写一堆用例，跑到一半发现环境不稳定， 然后花大量时间分辨「是我的用例错了还是环境抖了」。靶场稳定之后，失败就是真的失败。

设计原理 · 这套框架为什么长这样

这一节**不新增任务**，只回答一个问题： `gitlab-qa` 为什么是现在这个结构。 看完你应该能做到三件事——不看代码画出分层图；说出每个设计决定要防的是什么； 在面试里讲清取舍，而不只是罗列"我用了 pytest + requests + Playwright + Allure"。

◎ 总纲：框架的第一性目标

测试框架的第一性目标**不是"让用例跑起来"**，而是让三种信号可信：

信号	要求	做不到时的典型症状
红	可定位	失败只有一行 `assert 403 == 401`，必须本地复现一遍才知道哪个请求出的问题
绿	非假绿	报告全绿，实际一条需要鉴权的用例都没跑；越权断言恒真，永远不可能红
结论	可消费	别人拿到报告还得问你"这些红的是环境问题还是真 bug"

下面所有设计——分层、守卫、数据工厂、留证、套件划分——都是这三条的展开。**判断一个改动该不该做，也回到这三条。**

一 · 六层结构与依赖方向



上层可以调用下层，下层永远不知道上层的存在。判断某一行代码该放哪层的标准只有一句： **换一个被测系统，这一行还成立吗**——成立的往上放，不成立的往下沉。

层	换被测系统时	违反后长什么样（本项目的真实代价）
L1	**不该改**	用例里写死 `/api/v4/projects` 和 CSS 选择器 → 接口路径一变要改 N 个文件；GitLab 升版后 UI 用例成片红
L2	只改业务夹具	守卫写在用例里 → 有的用例 prod 下照写不误；清理散在各处 → 数据互相污染
L3	整体替换	`Projects._enc` 处理 `namespace/project` 的 `%2F` 编码——散在用例里的话，每个调用点都会踩一次
L4	基本不动	429 退避、分页、重试策略各写各的 → 有的用例把 GitLab 压到限流，账号被封
L5	改配置与契约	配置散落各处 `os.getenv` → 换环境改漏一处，出现"明明改了却没生效"
L6	只改地址	没有统一入口 → 每人跑出来的报告不一样，互相没法比对

二 · 八条设计原则

每条原则都有**代码落点**和**不这么做的代价**。只记住结构讲不出代价，面试一追问就露馅。

1 配置单一来源，且缺配置要早失败、说得清

加载顺序固定为 **环境变量 > .env > 默认值**，且 `.env` 按 `PROJECT_ROOT` 定位而不是靠当前工作目录—— 否则换个目录执行就读不到配置，症状是"明明配了却说没配"。

落点 `settings.py`：`load()` 唯一入口 · `has_token` / `has_member_token` 用 `<` 前缀判空占位 · `describe()` 脱敏输出（只显示首尾几位）

代价 缺 Token 时不早失败，会跑到第 40 条用例才因为 URL 是空值报一个看不懂的错

2 用例里不许出现裸路径和裸选择器

用例只跟"资源"和"页面对象"打交道：`gl.projects.create(...)`、`login_page.login(...)`。 接口路径、URL 编码、DOM 选择器全部分别收敛到 `resources.py` 和页面对象里。

落点 `GitLab` 门面（System/Projects/Issues/Users/Groups/Repository）· `Projects._enc()` 编码 · `BasePage._first([...])` 多候选选择器

代价 改一处接口动 N 个用例；GitLab 各版本 DOM 变化会让 UI 用例集体变红，而实际功能没问题

3 对假绿零容忍 —— 本框架最硬的一条

一个**永远不会红的断言**比没有断言更危险：它在报告上是绿的，会让你以为这条路径已经被守住。 本项目里真实出现过一次：越权用例用管理员访问管理员端点，永远返回 200，断言恒真。

落点 缺 Token / 缺第二身份 → `pytest.skip` 而不是 pass · `--strict-markers`（marker 拼错直接报错，而非静默不生效）· 全局 `addopts` 里**不放** `--reruns` · `run.py` 拦截 pytest 退出码 5（零收集）· 越权断言强制换身份

代价 报告全绿但一条关键用例都没跑 —— 这比全红危险得多，因为它会一直绿下去

4 数据自治：自己造、自己清、清理不许失败

测试数据带 **run-scoped 前缀**（前缀 + 随机后缀 + 自增计数），多轮运行、多人并行都不撞名； 创建即登记，按依赖倒序清理（议题 → 项目 → 群组），删除接受 `202/204/404` 三种结果——已经没了也算成功。

落点 `factory.py`：`ledger` 台账 · `cleanup()` 倒序 + 幂等 · `projects.purge()` 两步永久删除 · 清理结果作为 Allure 附件「数据清理记录」

代价 GitLab 默认延迟删除会把项目保留若干天（本机实测 30 天，最小只能设 1），跑几十轮就堆上百个僵尸项目，项目列表越来越慢，最终表现为 UI 用例**莫名其妙 timeout**——排查时没人会想到是清理方式的问题（本项目真实发生过：88 个待删除僵尸）

5 异步用轮询，不用 sleep

删除是后台任务、搜索要建索引、状态流转有延迟。固定 `sleep` 的取值必然在快环境浪费时间、在慢环境假失败。

落点 `utils/wait.py`：`wait_until(谓词, 超时, 间隔)` · `wait_for_status(请求, 期望状态码)`；期间的异常（如 404）视为"还没满足"继续等

代价 几百条用例各 `sleep(3)`，一轮跑几十分钟；CI 上偶发红，本地重跑又绿，从此没人信这个套件

6 失败即证据：三级留证自动化

留证**不能靠用例作者记得写**——统一挂在 `pytest_runtest_makereport` 钩子里，失败时自动attach，用例代码里一行都不用加。

落点 UI：截图 + DOM 快照 + `trace.zip` · API：完整请求/响应日志（含方法与耗时） · 性能：采样明细 markdown · 数据：清理台账

代价 失败只剩一行 assert，排查必须本地复现一遍；偶现失败永远查不出来

7 运行可复现：换个人跑出同样的结果

一套框架真正难的不是写用例，是**换个人、换台机器也能跑出同样的结果。**所以参数、结果目录、报告生成、趋势继承全部固化进一个入口。

落点 `tools/run.py`：9 个套件固化路径与并行度 · `-p no:base_url -p no:playwright` 屏蔽抢注插件 · `--mock` 起内置靶机（无需 Docker） · 清结果时**保留 history**（趋势才能连起来） · `--keep-results` 供安全策略禁止批量删除的环境

代价 本机装了 `pytest-base-url` / `pytest-playwright`，会抢注 `--base-url` 和 `page` 固件，pytest 在**收集前**就崩；换解释器跑又正常——排查时先看 `sys.executable`

8 门禁分层：不是所有测试都该挡住合并

门禁的价值 = **拦住真问题的概率 ÷ 它让人等待的成本。**所以按"快且稳"到"慢或噪"分层放。

落点 PR 门禁：`unit + smoke + api`（快、以只读为主） · 合并前：`ui + e2e` · 夜间 + 发版前：`perf + security` · `destructive` 在 `--env prod` 下**收集期就跳过**

代价 性能受宿主机噪声影响（本项目实测：并发清理 30 个项目时容器 CPU 打满，P95 从几百毫秒翻到 2 秒以上），放进 PR 门禁只会换来一堆无意义的红，最后大家学会无视它

→ 三 · 一条用例的完整链路

下面是一条 API 用例从被收集到进报告的七个环节。能默出这条链路，就知道该在哪里加断言、在哪里加留证。



③ 的重试只对 GET/HEAD/OPTIONS 生效：POST 重试可能造出重复资源，这是"重试必须区分幂等"的硬规则
⑤ 无论用例通过还是失败都会执行——清理挂在 fixture 的 teardown 上，不挂在用例里

四 · 契约为什么做两层

MODELS.PY · PYDANTIC

- 代表**我这边的期望**：`extra="allow"` 放行多余字段
- 只约束关键字段与类型
- 目的：我依赖的字段还在、类型没变
- 用途：用例里拿 `Project.id`、`Issue.is_open` 这类属性做业务断言

CONTRACTS.PY · JSONSCHEMA

- 代表**服务端的承诺**：`required` + `type` + `enum` 严格
- 一次列出所有不合约字段，而不是只报第一个
- 目的：上游结构漂移能报警
- 用途：契约测试独立成套件，可打公网实例做漂移检测

> **为什么两套都要**
> 只留 pydantic：
> **放不过类型漂移**
> ——上游把 `id` 从 int 改成字符串，宽松模型照样解析成功， 但所有按 id 拼路径的用例都会悄悄错位（`contracts.PROJECT` 里对 `id` 的类型约束就是为这个写的）。
> 只留 jsonschema：上游
> **多给一个字段就红一片**
> ，噪音太大，真问题反而被淹没。

五 · UI 层为什么"少而准"

UI 用例是全套里**最贵也最脆**的一类：慢、依赖浏览器、受页面微调影响。所以它只守关键路径，靠四件事压住不稳定性。

- **多候选选择器**：`_first([...])` 按优先级尝试，GitLab 各版本 DOM 差异不需要改用例
- **浏览器通道回退**：msedge → chrome → chromium。内网下载不到 Playwright 浏览器时，用系统已有的 Edge/Chrome 也能跑（真实约束：企业环境 CDN 常不通）
- **每条用例独立 context** + 全程 trace：cookie / localStorage 不串味，失败能回放每一步而不是只有一张截图
- **断言有效性**：`is_logged_in` 判定"可见的已登录元素"，而不是"元素存在即通过"——折叠菜单里那个链接一直存在但不可见，用存在性判定会得到假绿

> **验收标准**
> 每条 UI 用例都要能演示
> **「改坏就变红」**
> 。演示不出来的，说明这条断言没在守任何东西。

六 · 性能为什么是"门禁"而不是"图表"

- **固定样本数 + 并发**：不是"跑多久"，而是"取多少个样本"，结果才可比对
- **P95 用线性插值**：与 Prometheus / Grafana 口径一致，方便和线上监控对账
- **为什么不用平均值**：均值会被少量极慢请求掩盖；P95 对应"绝大多数用户的体感上限"
- **门禁必须自证有效**：用 `--mock-latency-ms` / `--mock-error-rate` 注入延迟和 503，**确认门禁真的会红**——没被验证过的门禁等于没有门禁

- **不进 PR 门禁**：宿主噪声大，且需要可比对的历史趋势才有意义

七 · 安全层的设计边界（已落地 32 条）

> **信任边界写进代码，不写在文档里**
> 安全用例
> **只对自己拥有的本地实例执行**
> 。`src/gqa/security/guard.py` 在每个用例前判定： 目标 host 不在白名单就
> **skip**
> ；需要指向别的机器时必须用 `SECURITY_ALLOWED_HOSTS` 显式授权。
> 这不是形式主义——自动化安全测试一旦能指向任意地址，性质就变了，而"记得别配错"不能算防护。

- **可回归优先**：一次扫描的结果不是资产，**能反复跑并断言的才是。**所以只做四类：越权、注入与转义、敏感字段泄露、暴露面开关
- **越权必须双身份**：`GITLAB_MEMBER_TOKEN` 提供普通用户视角；缺则 skip。用管理员访问管理员端点永远 200，断言恒真
- **403 与 401 要分清**：401 是没认证，403 是认证了但没权限。越权用例断言的是 403——写成 401 说明你测的是鉴权而不是授权
- **探针先手测再写用例**：`/admin/*` 是 Web 路径不是 API（真实返回 404），管理员端点要用 `/application/settings`。**mock 里通过不代表真实实例通过**

八 · 三个主动取舍（面试最容易加分的部分）

取舍	理由
自研 client 不用现成 SDK	要把 429 退避、分页、计时、请求日志这四件事握在自己手里——它们恰恰是"测试客户端"和"普通脚本"的分界线。现成 SDK 的重试与日志策略不为"测试要留证"设计
自写并发采样 不用 locust	这里要的是**能从 pytest 调用、能产出结构化结果、能让 CI 判通过/失败**的门禁，不是压测平台。locust 的产出是给人看的报表，不是给流水线用的退出码
不做全局失败重跑	`--reruns` 会把偶现的真缺陷重跑成绿。偶现失败是**信号**不是噪音——正确做法是标记 flake、留证据、单独治理，而不是让它消失

> **面试怎么讲这套设计**
> 按
> **目标 → 分层 → 防假绿 → 门禁**
> 的顺序讲，每讲一条附一个你真实付出的代价 （僵尸项目把 UI 拖超时 / 越权断言恒真 / 插件抢注导致收集前崩溃 / 退出码 5 伪装成通过）。
> **讲得出代价的设计才是你自己做的；只讲得出结构的多半是抄的。**

0 地基 · 拥有你自己说了算的靶场

目标：一台随时能起、能重置、数据随便造的 GitLab CE。这不是前置琐事，它决定了后面每一步的可信度。

W0 一键部署与验收

约 0.5 天

核心认知： 测试框架的价值，取决于靶场是否稳定。靶场不稳时，你会把环境问题误判成框架问题—— 这是新手浪费时间最多的地方（502、迁移卡住、服务没起来都可能伪装成"用例不稳定"）。

动手

- **一键部署**：`.\deploy.ps1` 跑完 8 个阶段（启动 → 等就绪 → 签发 PAT → 写 .env → 冒烟）
- **浏览器验证**：`py -3 tools/verify_browser.py` 走 打开→登录→进项目页 三步
- **环境自检**：`py -3 tools/doctor.py` 查连通性/鉴权/分页/写权限
- **离线兜底**：`python tools/run.py --suite all --mock` 用内置靶机跑，不依赖真实例

产出物

- `gitlab-ce/docker-compose.yml` — 关掉监控栈省内存
- `gitlab-ce/deploy.ps1` — 8 阶段一键脚本
- `gitlab-qa/.env` — 12 个配置键（含 PAT）
- `gitlab-ce/logs/browser-0*.png` — 浏览器实测证据

已踩过的坑（都已修进脚本，你要知道为什么）

① GitLab 19 **移除了** `grafana['enable']`，写了就 `UnknownConfigOptionError` → 容器无限重启； ② 19 版健康端点变了，`/-/readiness` 返回 404，就绪判定要换成「sign_in=200 且匿名 API 有响应」； ③ 初始密码 `GitlabQA@2026` 被密码策略 **静默拒绝**，root 账号根本没建出来——表现为"部署成功但登录不了"； ④ PowerShell 写 `.env` 必须 **无 BOM**，否则第一个键名带 `\ufeff`，dotenv 读不到。

验收标准 0/4

- [ ] 浏览器能打开 http://localhost:8929 并用 root 登录
- [ ] `verify_browser.py` 输出 7/7 PASS
- [ ] `doctor.py` 全绿，`--suite smoke` 通过
- [ ] 能说出"靶场不稳"和"用例失败"的区别，并能用 doctor 分辨

1 框架搭建 · 让它撑得住 200+ 条用例

目标：不是"能跑"，而是"加 100 条用例后依然清晰、失败能定位、改一个地方不会全崩"。

W1 pytest 工程化

夹具分层 · 作用域 · marker · 配置

核心认知： 框架的地基是**配置单一来源 + 夹具分层。**配置散落在各处 `os.getenv`， 换环境时改到漏一处就出现"明明改了却没生效"；夹具不分层则每个测试文件都会长出一堆重复 setup。

动手

- **三层 conftest**：根（全局夹具 + 环境守卫）/ 包（`tests/conftest.py` 共用夹具）/ 模块级
- **作用域验证**：用**构造计数器**验证，不要用 `id()`
- **marker 分组**：10 个 marker + `--strict-markers`
- **配置收敛**：`.env → settings.py`，其它地方一律 import

为什么这么做

- `--strict-markers`：写错 marker 直接报错，避免**打了标签却没生效**的假分组
- 环境守卫：prod 环境下自动跳过 `destructive` 用例
- `requires_token`：缺 Token 时 **skip 而不是 pass**——假绿比失败危险

```
# 验证 fixture 作用域：用计数器，别用 id()
# CPython 会复用已释放对象的内存地址，id() 相同不代表是同一个实例，
# 用它验证会得出「作用域正确」的错误结论。
created = {"n": 0}

@pytest.fixture(scope="session")
def client():
    created["n"] += 1
    return ApiClient(...)

def test_scope_once(client):
    assert created["n"] == 1   # 第二次调用不会再造
```

两个绝对不能犯的错

① `--reruns`

绝不能

放进全局 `addopts`——它会把真 bug 重跑成绿，你永远发现不了那个偶现缺陷； ② 插件冲突：本机装了 `pytest-base-url` / `pytest-playwright`，会抢注 `--base-url` 选项和 `page` 固件， pytest 在

收集前

就崩，报「conflicting option string」。已在 `addopts` 固定 `-p no:base_url -p no:playwright`。

这个坑的迷惑性

托管的 Python 没装这两个插件，同一个仓库换解释器跑会"一边报错一边正常"。 排查同类问题时先看 `sys.executable`，别急着改代码。

W2 数据工厂与失败留证

造得出来 · 清得干净 · 挂了能查

核心认知： 测试数据的标准是「**自己造、自己清、清理不许失败**」。 清理失败比不清理更糟：它会让整批用例在第二次运行时全红，而你以为是新 bug。

动手

- `factory.py`： `yield` 造数据 + `finally` 清理
- 清理要**幂等**：接受 `204 或 404` （已经没了也算成功）
- Allure 附件：失败时自动附上请求/响应/截图
- UI 失败挂 `trace.zip` （放在 fixture teardown 里）

ALLURE 两个隐性坑

- `environment.properties` 按 **ISO-8859-1** 读取，中文必须转 `\uXXXX` （见 `allure_meta.py` ）
- `categories.json` 用 Java `String.matches()` **全串匹配**， `.` 不跨行，多行堆栈要加 `(?s)`
- `history/` 目录驱动趋势图，清理结果时**必须保留**，别用 `--clean-alluredir`

```
# 清理幂等：目标可能已被上游删掉，404 也要当成功
# 否则「用例通过了但 teardown 报错」，污染的是整批结果的判定
@pytest.fixture
def project(api):
    pid = api.create_project(f"qa-auto-{uuid4().hex[:8]}")
    yield pid
    try:
        api.delete_project(pid)
    except ApiError as e:
        if e.status not in (204, 404):
            raise
```

阶段一验收 0/5

- [ ] 能解释 conftest 三层各自放什么，并说清为什么不放一起
- [ ] 能用计数器证明 session/function 作用域的行为差异
- [ ] 造数据的用例连续跑 3 次，实例里不留垃圾
- [ ] 故意让一条用例失败，Allure 里能看到请求/响应证据
- [ ] Allure 趋势图能跨两次运行连起来（history 没被删）

2 API 测试 · 从"能调通"到"能抓到服务端撒谎"

目标：大部分人停在"断言 200 和几个字段"。这层的真本事是**契约校验**和**鉴权状态机**。

W3 客户端与鉴权

会话 · 超时 · 重试 · 错误分类

动手

- `client.py`：会话复用、显式超时、错误分类（网络 / 4xx / 5xx）
- **鉴权状态机**：登录 → 过期 → 透明续期 → refresh 一次性 → logout 失效
- Token 分级：管理员 / 只读，用不同夹具
- 断言错误响应体格式一致（不只看状态码）

真本事在哪

- 能测出「**Token 过期后没正确续期**」这类缺陷，而不是只会测正常路径
- refresh token **只能用一次**——重复用应失败，这是常见安全缺陷
- logout 后旧 Token **必须失效**，仍能用是严重问题

最隐蔽的一个 bug（本项目真实踩过）

服务端返回 401 时客户端
不读请求体
，keep-alive 上残留的字节会顶掉下一个请求， 表现是「随机失败」——同样的用例重跑就好。反过来，业务已读 body 后错误处理再读一次就会阻塞 20 秒。
修法是让 drain 幂等
（用 `_body_read` 标记），drain 与解析共享同一次读取。 同类问题排查要点：
偶发 + 重跑就好 = 先怀疑连接复用，不要先怀疑被测系统
。

W4 契约 · 分页 · 数据依赖

三层 schema 校验

核心认知： 契约校验的价值在于抓住「**服务端悄悄改了字段类型**」—— 这种改动不会让接口报错，但会让接入方全线崩溃。只断言"字段存在"是抓不到的。

层	工具	校验什么	能抓住什么
本地期望	`pydantic`	我们依赖的字段是否符合预期	字段缺失、结构变形
服务端承诺	`jsonschema`	响应是否符合文档声明	**类型漂移**（见下方实测）
发布契约	OpenAPI	文档与实现是否一致	文档撒谎、版本不兼容

实测教训：pydantic 会放过类型漂移

lax 模式下 `"1991"`（字符串）被自动转成 `1991`（整数）并**校验通过**。也就是说：服务端把年份从数字改成了字符串，你的接口测试**依然是绿的**，而下游已经炸了。只有 jsonschema 严格模式会拦住它。这就是必须做三层而不是一层的原因。

分页测试要点

- 遍历全部页，校验**无重复、无遗漏**
- 边界：`per_page=1`、超出范围的 page、`page=0`
- 分页期间数据被删 → 不应 500

数据依赖原则

- 用例**不能假设数据已存在**，自己造自己清
- 只读用例可以依赖种子数据，但断言必须**能失败**
- 用例间禁止顺序依赖（`-p no:randomly` 也要能过）

阶段二验收 0/4

- [ ] 写得出鉴权状态机全套用例（含过期/续期/一次性/失效）
- [ ] 能演示一次「pydantic 放过、jsonschema 拦住」的类型漂移
- [ ] 分页用例能抓出重复或遗漏（造 25 条数据验证跨页）
- [ ] 随机打乱用例顺序，结果一致

3 UI 测试 · 少而准，只守关键路径

目标：UI 用例不是越多越好。这层的关键能力是**判断哪些该用 UI 测**，以及**断言真的有效**。

W5 Playwright 与页面对象

POM · 留证 · 断言有效性

动手

- **POM 分层**：`pages/` 只暴露业务动作，不暴露选择器
- **通道回退**：msedge → chrome → chromium
- 失败自动挂 `trace.zip` + 截图
- `destructive` 标记：prod 环境自动跳过

为什么

- 选择器散落在用例里 = 改版时全量重写；收进 POM 只改一处
- 企业内网下 Chromium CDN 下不来，**本机 Edge 一定有**，不需要下载
- trace 能回放失败瞬间的完整 DOM 和网络，比截图有用得多

最重要的一条：你的断言必须「能失败」

本项目真实踩过：匿名访问项目列表的用例 **通过了**，但它是无意义的—— 因为种子项目全是 private，匿名看到空列表，断言「返回的是列表」恒真。

判据：把被测功能弄坏，用例必须变红。

做完后故意改坏一次验证（这叫变异测试）。

UI 覆盖面的取舍

业务组合的验证放 API 层（快、稳），UI 只守「用户真正会点」的关键路径： 登录、创建项目、提 issue、搜索。一条 UI 用例的维护成本约等于十条 API 用例。

阶段三验收 0/4

- [ ] POM 里找不到任何裸露的 CSS 选择器
- [ ] 每条 UI 用例都能演示「改坏后变红」
- [ ] 失败时能在 Allure 里打开 trace 回放
- [ ] prod 环境跑 `--env prod` 时写操作用例自动跳过

4 性能测试 · 有阈值的门禁才算数

目标：跑出数字不难，难的是**这个数字能拦住退化**。没证明过自己有效的门禁等于没有。

W6 并发采样与门禁

P95 · 错误率 · 有效性验证

动手

- `perf/runner.py`：并发采样 → 统计 P50/P95/P99 + 错误率
- 阈值从 `.env` 读：`PERF_P95_MS` / `PERF_ERROR_RATE`
- 超阈值 → 退出码非 0 → CI 阻断
- 结果入 Allure，能看趋势

关键认知

- **均值没用**，看 P95/P99——均值会掩盖长尾
- 性能**不进 PR 门禁**，走夜间：单机结果噪声大，误报会毁掉门禁可信度
- 必须**证明门禁有效**：注入一个慢接口，门禁必须报警

反直觉实测：并行反而更慢

本仓库实测：单进程 **2.78s**，4 worker **6.05s**。短用例下进程调度 + 结果回传的开销大于并行收益。**别盲信 xdist**，用数据决定 `workers`——套件定义里每个套件单独配了 worker 数就是这个原因。

阶段四验收 0/4

- [ ] 能说出为什么用 P95 而不是平均值
- [ ] 人为注入慢接口，性能门禁确实失败（证明其有效）
- [ ] 能解释为什么性能不放在 PR 门禁里
- [ ] 性能结果在 Allure 里能看到趋势变化

5 安全测试 · 把一次性扫描做成可回归资产 已落地 · 32 条

`tests/security/` 已建成，覆盖**越权 / 注入转义 / 信息泄露 / 暴露面**四类， 独立成套件并接进 CI 夜间 job。它是面试时最能拉开差距的一层—— 因为它需要你同时懂权限模型、HTTP 语义和"怎么让断言不作假"。

授权边界（必须严格遵守）

以下所有用例**只对你自己部署的本地 GitLab CE 实例执行**。对不属于自己的系统做同样操作是违法行为。这套计划的合规性建立在"靶场是你的"这个前提上。

W7 四类可自动化安全用例

越权 · 注入 · 泄露 · 暴露面

核心认知： 安全测试的价值不在"扫出漏洞"，而在**把安全约束变成可回归的断言**—— 每次发版都自动验证一遍"普通用户还是进不了后台"。一次性扫描报告会过期，可回归用例不会。

A · 鉴权与越权（最高价值）

用例	操作	期望
对照组：身份有效性	第二身份请求 `/user`	200 且**不是管理员**
普通用户读管理端点	非管理员 Token 请求 `/application/settings`	**403**
跨用户私有项目	普通用户读管理员的私有项目	**404**（隐藏存在性）
跨用户删除	普通用户删管理员的项目	403/404，且**确认项目还在**
跨用户写入	普通用户向他人项目提 issue	403/404，且**确认未落库**
字段级越权	普通用户读管理员用户档案	200 但**不含 email**
未鉴权访问	不带 Token 请求需鉴权接口	401

403 与 404 的分工（这里很容易写错）

403

= 资源本身存在且可见，但你不能做这个操作（例：管理端点对普通用户）。

404

= 连"这个资源存在"都不该告诉你（例：别人的私有项目）。 对私有资源用 403 反而**泄露了存在性**

——攻击者可以靠 403/404 的差异枚举项目名。

所以断言口径是：

不能是 200，具体 403 还是 404 取决于资源是否公开

。 一刀切写 `== 403` 会误判，写 `in (403,404)` 才是对的。

对照组是必须的（写完越权用例才发现）

如果 `GITLAB_MEMBER_TOKEN` 过期，所有越权用例都会收到 401， 而 401 在"允许的状态码"里 → 全套照绿。所以必须先有一条 "第二身份可用且非管理员"的用例当前置断言。没有它，这个套件就是最危险的那种绿。

探针端点必须先在真实实例上验证存在（本项目实测踩过）

原先用例用 `/admin/users` 做管理员探针，结果

管理员访问也返回 404

—— 那是 Web 路径，不是 REST API 端点。它在内置 mock 靶机上"通过"，到真实 GitLab 19 上必然失败。

正确探针：`/application/settings`（管理员 200、普通用户 403、只读且真实存在）。

教训：mock 通过 ≠ 真实通过。新探针先用 curl/API 手测一次再写进用例。

B · 注入与转义（12 条）

- **存储型 XSS**（3 条）：把 `<script>` / `<img onerror>` / `javascript:` 写进 issue 描述，读回断言**原样保留**，且响应 `Content-Type` 必须是 `application/json`。 接口层能断言的边界只到这里——"浏览器里会不会执行"是 UI 层的事，写在这里就叫假测试。
- **模板注入 SSTI**（3 条）：写 `{{ 7 * 7 }}` / `<%= 7 * 7 %>` / `${7*7}`， 读回**仍含载荷原文、且不含 49**。出现 49 说明服务端把用户输入送进了模板引擎，危害远大于反射型 XSS。
- **SQL 注入**（3 条）：搜索参数传 `' OR '1'='1` / `'; DROP TABLE` / `UNION SELECT`， 断言**不是 500**、正文无 SQL 报错特征、返回条数没有异常放大（防条件恒真）。
- **路径穿越**（3 条）：**必须用原始请求发**——原因见下面这条实测。
- **命令注入：没做。** GitLab 没有可拼接 shell 的入口，硬造只会得到一条永远绿的空用例。

实测：路径穿越差点测了个寂寞

`requests` / `urllib3` 会在发送前把 `../` 归一化掉， `%2e%2e%2f` 也会先 unquote 再消解——**恶意路径根本没到服务端**，返回的是 nginx 的 HTML 登录页。用例"通过"只是因为载荷没发出去。

必须用 `http.client` 直接写请求行（`src/gqa/security/raw.py`）。 判定口径也要放宽：**4xx 算拒绝；2xx 也不算漏洞，只要正文里没有 `root:x:0:0` 这类文件特征**

—— 反代把路径归一化到登录页，本来就不是缺陷。

C · 信息泄露（8 条）

- **敏感字段键名黑名单**：扫响应 JSON 的**键名**，不扫值——值里出现 `token` 太常见（URL、分支名、描述）， 误报会让人把整个安全套件关掉，那比不测更糟。
- **错误响应不吐堆栈**：400 / 401 / 404 三种都单独扫。正常响应大家都会查，**出错的路径才是最常泄露的地方。**
- **列表与详情分开看**：列表是批量暴露面（一次泄露 N 个项目）；详情按权限模型判定。
- **字段级越权**：普通用户能读用户档案，但不该看到 `email`。端点扫描器扫不出这类问题。

> **实测：runners_token 到底算不算泄露**
> 管理员读 `/projects` 列表，每个项目都带 `runners_token`。 这是**权限模型内**的行为（admin 对任何项目都有 maintainer 权限），报失败属于误报。 于是断言改成用**非成员视角**读同一个公开项目：拿到才是泄露——可直接注册 runner 截获 CI 任务。
> 同一轮还修掉两个误报：`ci_id_token_sub_claim_components`（配置项数组）、`ci_push_repository_for_job_token_allowed`（布尔开关）——名字带 token 但不是凭据。**扫描器的白名单和黑名单一样重要**，否则红几次就没人看了。

D · 暴露面与配置（6 条）

- **安全响应头**：`X-Content-Type-Options: nosniff`、`X-Frame-Options`（防点击劫持）。 HSTS 只在 https 目标上断言——本地 http 靶场没有是正常的，强断言就是自找假红。
- **公开注册**：断言 `signup_enabled=false`。**本轮实测就是开着的**，已关闭并回读确认。
- **匿名可见范围**：匿名能列项目不是问题，泄露私有项目才是——断言"每一条都是 public"，而不是"匿名不能访问列表"。
- **管理端点匿名不可达**：`/application/settings`、`/statistics`、`/users` 均须 401/403/404。
- **SSRF / 暴力破解：没做。** 需要外部回调地址和多次失败登录，属于 ZAP 这类工具的领域。 塞进 pytest 会变慢、变不稳，还会因为"等锁定生效"引入 sleep——正好违背本框架的原则。

> **本轮的真实战果：三个发现，只有一个是产品问题**
> ① `signup_enabled=true` → 任何人可自助注册，已用 API 关闭； ② 管理员项目列表带 `runners_token` → 记入报告"已知行为"，改用非成员视角断言； ③ 路径穿越被客户端归一化 → 补原始请求发送器，否则这条测试永远假绿。
> **③ 这类"测试自己的 bug"最值得讲**：它说明你有能力怀疑自己的工具，而不只是相信绿色的报告。

E · 依赖与镜像（未做，可选）

- `pip-audit` 扫 Python 依赖； `trivy` 扫容器镜像
- 结果作为 CI 制品保留，**高危才阻断**（全部阻断会淹没真实问题，最后没人看）
- 为什么先不做：它属于"供应链"而不是"被测系统"，与本套件的信任边界不同，分开跑更清楚

产出物（已建成）

- `tests/security/test_authz.py` — 越权 6 条
- `tests/security/test_injection.py` — 注入转义 12 条
- `tests/security/test_leak.py` — 信息泄露 8 条
- `tests/security/test_exposure.py` — 暴露面 6 条
- `src/gqa/security/` — `guard.py` 白名单守卫 / `scanner.py` 扫描器与载荷库 / `raw.py` 原始请求
- `tools/ensure_member.py` — 建第二身份并签发 Token
- 新 marker + 新套件：`security`（夜间 + 发版前）

工程要点

- 失败要**附请求证据**（根 conftest 已自动挂请求日志），不能只写"断言 403"
- 必须**两种身份**：管理员 + 普通用户，且先验证第二身份可用
- 独立 job，**不在 PR 门禁**：会写数据、耗时 30s+，且红了往往要人工判断
- **信任边界写进代码**：目标不在白名单 → skip，不靠人记得加参数

```
# 越权用例的写法要点
# 1) 只断言状态码，失败后你只知道"期望 403 实际 200"；
#    附上响应摘要，才能立刻判断是权限漏了还是接口改了
# 2) 注意本框架的 Response 是自定义 dataclass：
#    已解析的 JSON 在 .json_data 上，没有 requests 的 .text / .json()
def test_member_cannot_read_admin_settings(gl, member_client):
    assert gl.client.get("/application/settings").status == 200   # 管理员放行
    r = member_client.client.get("/application/settings",
                                 expect=(200, 401, 403))
    attach(f"status={r.status}\n{str(r.json_data)[:500]}", name="响应证据")
    assert r.status == 403, f"越权成功！HTTP {r.status}"
```

最小示范已经做完了

实例原本 `signup_enabled=true`（任何人可自助注册）。 先关掉（ `PUT /application/settings?signup_enabled=false` ），

再写一条用例断言它必须保持关闭

——这就是"把安全约束变成可回归资产"： 三个月后有人为了拉人把它打开，这条用例会红。

阶段五验收 0/5

- [ ] 建立 `tests/security/`，四类各至少 2 条用例
- [ ] 造出第二个普通用户，能验证跨用户越权
- [ ] 安全用例失败时 Allure 里有请求证据
- [ ] 关闭公开注册后，有用例断言它不能再被打开
- [ ] `--suite security` 可独立运行并接入 CI

6 回归策略 · 让"变红"这件事可信

目标：解决自动化测试最大的信任危机——**大家不相信变红是真问题。**

W8 分层门禁与 flake 治理

分类 · 度量 · 可信度

触发点	跑什么	时限	失败意味着
每次 commit	`unit`	< 30s	框架自身坏了
PR 门禁	`smoke` + `api`	< 5 min	核心功能退化，**禁止合并**
合并前	`regression`（含 ui/e2e）	< 20 min	业务流退化
夜间	`all` + `perf` + `security`	不限	性能退化 / 安全约束被破坏

四类结论，不是两类（已落地）

判定	含义	该做什么
`STABLE`	N 轮全绿	结论可信
`BROKEN`	N 轮全红	**这是缺陷，不是抖动**——别往隔离区丢
`FLAKY`	有绿有红	抖动率 = 红/（绿+红），留证据后限期隔离
`NO_DATA`	N 轮全是 skip	**等于没测**，算成"稳定"是最危险的假象

最后一类最容易被漏：因缺 Token 而 skip 的用例，在报告里和通过长得一模一样。 区分不了这两件事，稳定性度量就是自欺欺人。

失败分类（CATEGORIES.JSON）

- **产品缺陷** → 找开发
- **环境问题** → 找运维 / 查靶场
- **测试脚本问题** → 自己改
- **基础设施** → CI runner / 网络

为什么分类这么重要

一片红的时候，第一问题是"该找谁"。 Categories 让你 10 秒内判断该找开发还是查环境——这是**工程成熟度的直接体现**， 也是面试时讲"我怎么让团队重新相信自动化"的核心素材。

落地形态 · `tools/flake.py` + `src/gqa/flake/`

一轮跑绿说明不了什么，能重复 N 轮都绿才叫稳定。采样器把一套用例重复跑 N 轮， 逐条给出上面的四类判定——**不是"通过率"，是每条用例的可信度画像。**

```
# 重复 5 轮采样。轮次是有代价的选择：5 轮能把 20% 的抖动抓出来，
# 想证伪 1% 的抖动要上百轮，成本远高于收益—默认值取 5
python tools/flake.py --suite smoke --rounds 5

# 实测输出（4 轮，含一个故意写出的演示文件，验完即删）：
#   FFFF   100%   持续失败   ← 真缺陷，不是"100% 抖动"
#   FPFP    50%   抖动       ← 只有这一条才该进隔离区
#   SSSS     0%   无有效数据 ← 全程 skip，不能算稳定

# 门禁：单条抖动率超过阈值就退出非 0（退出码 2=持续失败 / 3=抖动超阈值）
python tools/flake.py --suite api --rounds 7 --max-flake-rate 0.05

# 隔离区体检：CI 每次提交都跑
python tools/flake.py --check-quarantine --verify-nodeids
```

隔离区是收费的

- 每条必须有 `reason` 和 `due`，缺一样 CI 就红
- **到期自动失效**，用例回到门禁里继续跑——不可以自动续期
- `--include-quarantine` 强制连它们一起跑，验收修复是否真生效
- 规模超 8 条报警：否则"临时豁免"会变成常态
- 核对名单里的用例是否还存在（改名后记录会过期）

重试必须上报

- 框架**不做全局失败重跑**：偶现失败是信号，重跑成绿等于消灭信号
- 只有显式 `@pytest.mark.flaky` 的用例才重试
- 次数写进 Allure 附件 + `artifacts/flake/reruns.jsonl` 累积留痕
- 看得出"这条本周靠重试混过去 6 次了"——这才叫可审计

三个踩过的坑

- **nodeid 必须对得上**
  ：JUnit 把模块写成包路径、把类塞进 `classname`， 拼错不会报错，只表现为"加了隔离却照跑不误"
- **N 轮 × Allure 会毁掉趋势**
  ：同一批用例重复写进主结果目录， Allure 会把一次采样当成 N 次运行——采样只留 junit，不写 alluredir
- **Windows 上 `os.environ[k] = ""` 等于删除变量**
  ：跑靶机时想停用第二身份， 写空串后 `load_dotenv` 又把真 Token 填回来。必须用非空占位符

> 隔离区闭环已实测 10 步：加入 → 跳过（reason 带期限/负责人/原因）→ 强制包含恢复执行 → 到期自动失效 → 体检报出「缺期限 / 已到期 / 用例不存在」并退 4 → 移出 → 空名单体检通过。

阶段六验收 **0/4**

- [ ] 能说清每个触发点跑什么、为什么这么分
- [ ] 四类失败都能在 Allure Categories 里正确归类
- [ ] 有一条被标记为 flake 的用例，并给出抖动证据
- [ ] PR 门禁能在 5 分钟内跑完并阻断合并

运行产物治理 · 定时清日志 / 截图 / 报告

测试跑得越久，磁盘越可能先扛不住。四类产物增长方式不同，但都会堆：

目录	谁写的	增长速度
`gitlab-qa/artifacts/`	UI 用例	每条用例一个 `trace.zip`（1–5 MB），失败再加截图 + DOM 快照
`gitlab-qa/allure-results/`	pytest	每次运行一批 JSON + 附件
`gitlab-qa/allure-report/`	`allure generate`	可重建，但会留多份
`gitlab-ce/logs/`	`deploy.ps1` / `verify_browser.py`	每次部署一份日志 + 若干截图

三条保留规则（同时生效，取最保守）

- **按时间**：只删 N 天前的（默认 7）
- **按体积**：单目录超 M MB 时从最旧删到阈值内（默认 1024）
- **按数量**：每个目录至少留最新 K 个（默认 3），防止清空

两个东西永远不删

- `allure-results/history/` —— 趋势图的原料，删了报告永远是孤立快照
- `allure-report/history/` —— `run.py` 从这里继承上一次的历史
- 顺带保护 `.env` / README（配置不能被"清理"掉）

```
# 先预览：这一步不删任何文件
# 实测输出：删除 42 个 / 70.5 MB，保留 3 个，失败 0
python tools/cleanup.py --dry-run

# 按 .env 策略执行（CLEANUP_* 四个键）
python tools/cleanup.py

# 临时改策略：只清日志、只留 3 天
python tools/cleanup.py --targets logs --days 3

# Windows：注册每日 03:00 计划任务（时间可配置，策略写回 .env）
powershell -ExecutionPolicy Bypass -File tools\schedule_cleanup.ps1 -Time 03:00 -Days 7

# Linux / macOS：打印 crontab 行
python tools/cleanup.py --print-cron
```

> **为什么逐文件删，而不是 `rm -rf`**
> 公司终端和带 DLP 的机器会拦批量删除，甚至在删除过程中把进程干掉—— 表现是"脚本跑了，但什么都没删"，而且不报错。 所以 `cleanup.py` 逐个 `unlink`、单个失败只记录不中断，最后把失败原因打出来；
> **有失败就返回非 0**
> ，让计划任务显示异常，而不是静默失效。 删除前还会校验路径必须落在目标目录内——防软链接把删除带到项目外。

CI 里另有两道防线：GitLab CI 的 `artifacts.expire_in`（3/7/30 天）由平台自动清； 定时流水线末尾的 `cleanup` job 负责清 runner 本地磁盘， 它挂在 `stage: .post` ——保证在所有测试之后跑，不会删掉本次刚产出的证据。

7 CI/CD · 测试结果要能自己说话

目标：本地跑通只完成了一半。另一半是**没人在场时，它也能跑、能报告、能拦住。**

W9 双平台流水线

GH Actions · GitLab CI · 趋势 · 门禁

动手

- `.github/workflows/ci.yml` + `.gitlab-ci.yml` **双跑**
- 分层 job：unit / smoke / api / ui / perf / security
- 制品归档 `allure-results`，报告发布 Pages
- 报告**汉化后处理**：`localize_report.py`

趋势怎么来的

- `history/` 目录跨次累积 → Allure 趋势图
- CI 里要**把 history 持久化**（cache / artifact），否则每次都是孤点
- 清理结果时保留 history，别用 `--clean-alluredir`

报告汉化的正确做法

Allure 2.29 **自带中文语言包**，只是 index.html 写死了 `<html lang="en">`。但 `allure generate` **每次都会重写 index.html**，所以汉化必须做成**生成后的后处理**，而不是改一次就完事 —— 见 `tools/localize_report.py`，已接进 run.py 和两条 CI。它只补默认值，不覆盖你手动选过的语言。

CI 里最容易翻车的三点

① **插件冲突**：CI 镜像装了 playwright 插件就抢固件名，固定加 `-p no:playwright`；② **工作目录**：subprocess 不继承 shell 的 cd，相对路径会解析错（本项目踩过，改成显式 WorkingDirectory）；③ **退出码**：门禁靠退出码阻断，报告生成失败却吞掉退出码 = 门禁形同虚设。

阶段七验收 0/4

- [ ] 两条流水线都能独立跑通并产出 Allure 报告
- [ ] 报告的语言是中文，且连续跑两次仍有趋势图
- [ ] 故意让一条用例失败，流水线确实被阻断（红叉）
- [ ] 安全套件作为独立 job 出现，夜间触发

8 面试转化 · 把工程经历讲成能力证据

目标：面试官不关心你写了多少条用例，关心你**踩过什么坑、怎么判断、怎么取舍。**

W10 怎么讲这个项目

STAR · 追问 · 简历

讲什么（按含金量排序）

- **分层设计**：为什么这么分，每层什么时候跑
- **踩坑与定位**：keep-alive 残留字节导致的随机失败
- **反直觉结论**：并行反而更慢，用数据决定 worker 数
- **有效性意识**：断言必须能失败；门禁必须证明有效
- **安全维度**：把安全约束变成可回归资产

不要讲什么

- "我写了 50 条用例" —— 数量不代表能力
- "用了 pytest + requests + Playwright" —— 这只是工具名词
- "CI/CD 都配好了" —— 要说清门禁**拦住了什么**
- 把环境问题讲成自己的功劳或别人的锅 —— 要说**如何分辨**

高频追问与回答方向

追问	你要答到的点
用例失败了你怎么排查?	先分类（产品/环境/脚本/基建），Categories 直接给出方向；偶发则先怀疑连接复用
怎么防止假通过?	缺 Token 时 skip 而非 pass；断言必须能失败；变异测试验证
为什么 UI 只测这些?	成本账：一条 UI 的维护成本≈十条 API，业务组合下沉到 API 层
性能阈值怎么定的?	P95 而非均值；阈值可调；**注入慢接口证明门禁有效**
自动化覆盖率多少?	不讲百分比，讲**分层门禁拦住了什么**——这比数字可信
怎么保证用例不互相影响?	数据自造自清、清理幂等、随机顺序仍能过

简历写法示例

基于 GitLab CE 搭建企业级自动化测试框架（pytest + requests + Playwright + Allure）， 设计 unit/smoke/api/ui/e2e/perf/security 七层测试体系与分级 CI 门禁； 通过契约三层校验拦截服务端字段类型漂移，建立安全回归套件将越权与暴露面约束固化为可执行断言。

阶段八验收 0/4

- [ ] 能在 3 分钟内讲清分层设计与取舍理由
- [ ] 能讲出 3 个自己定位并解决的真实坑（含根因）
- [ ] 能回答"怎么防止假通过"且不止一种手段
- [ ] 简历描述已改写成能力导向而非工具罗列

自检总清单

勾选状态自动保存到本机浏览器，换设备不丢失（清空需清 localStorage）。打印时会保留当前勾选。

全部验收项 0/38

验收标准 · 地基 · 拥有你自己说了算的靶场

- [ ] 浏览器能打开 http://localhost:8929 并用 root 登录
- [ ] verify_browser.py 输出 7/7 PASS
- [ ] doctor.py 全绿，--suite smoke 通过
- [ ] 能说出"靶场不稳"和"用例失败"的区别，并能用 doctor 分辨

阶段一验收 · 框架搭建 · 让它撑得住 200+ 条用例

- [ ] 能解释 conftest 三层各自放什么，并说清为什么不放一起
- [ ] 能用计数器证明 session/function 作用域的行为差异
- [ ] 造数据的用例连续跑 3 次，实例里不留垃圾
- [ ] 故意让一条用例失败，Allure 里能看到请求/响应证据
- [ ] Allure 趋势图能跨两次运行连起来（history 没被删）

阶段二验收 · API 测试 · 从"能调通"到"能抓到服务端撒谎"

- [ ] 写得出鉴权状态机全套用例（含过期/续期/一次性/失效）
- [ ] 能演示一次「pydantic 放过、jsonschema 拦住」的类型漂移
- [ ] 分页用例能抓出重复或遗漏（造 25 条数据验证跨页）
- [ ] 随机打乱用例顺序，结果一致

阶段三验收 · UI 测试 · 少而准，只守关键路径

- [ ] POM 里找不到任何裸露的 CSS 选择器
- [ ] 每条 UI 用例都能演示「改坏后变红」
- [ ] 失败时能在 Allure 里打开 trace 回放
- [ ] prod 环境跑 --env prod 时写操作用例自动跳过

阶段四验收 · 性能测试 · 有阈值的门禁才算数

- [ ] 能说出为什么用 P95 而不是平均值
- [ ] 人为注入慢接口，性能门禁确实失败（证明其有效）
- [ ] 能解释为什么性能不放在 PR 门禁里
- [ ] 性能结果在 Allure 里能看到趋势变化

阶段五验收 · 安全测试 · 把一次性扫描做成可回归资产 已落地 · 32 条

- [ ] 建立 tests/security/，四类各至少 2 条用例
- [ ] 造出第二个普通用户，能验证跨用户越权
- [ ] 安全用例失败时 Allure 里有请求证据
- [ ] 关闭公开注册后，有用例断言它不能再被打开
- [ ] --suite security 可独立运行并接入 CI

阶段六验收 · 回归策略 · 让"变红"这件事可信

- [ ] 能说清每个触发点跑什么、为什么这么分
- [ ] 四类失败都能在 Allure Categories 里正确归类
- [ ] 有一条被标记为 flake 的用例，并给出抖动证据
- [ ] PR 门禁能在 5 分钟内跑完并阻断合并

阶段七验收 · CI/CD · 测试结果要能自己说话

- [ ] 两条流水线都能独立跑通并产出 Allure 报告
- [ ] 报告的语言是中文，且连续跑两次仍有趋势图
- [ ] 故意让一条用例失败，流水线确实被阻断（红叉）
- [ ] 安全套件作为独立 job 出现，夜间触发

阶段八验收 · 面试转化 · 把工程经历讲成能力证据

- [ ] 能在 3 分钟内讲清分层设计与取舍理由
- [ ] 能讲出 3 个自己定位并解决的真实坑（含根因）
- [ ] 能回答"怎么防止假通过"且不止一种手段
- [ ] 简历描述已改写成能力导向而非工具罗列

使用建议：每周开始前先看该阶段的"核心认知"，它说的是**为什么**—— 只照做不理解为啥，面试一追问就露馅。每个阶段验收项全部勾选后再进入下一阶段。